

The transformer, mechanically

What each block does, what it costs, and how to count it by hand

Open the `config.json` of almost any open model on Hugging Face and you will find a short list of integers: `hidden_size: 4096`, `num_hidden_layers: 32`, `num_attention_heads: 32`, `intermediate_size: 11008`, `vocab_size: 32000`. That is essentially the whole architecture. Those five numbers determine that the model has 6.7 billion parameters, that its weights take 13.5 GB in 16-bit precision, that every token it processes costs about 13 billion floating-point operations, and that serving it with a 4,096-token context needs another 2 GB of memory per sequence just for attention. If you can look at that config and produce those four facts in your head, you understand the transformer well enough to train one efficiently. By the end of this chapter, you will.

We are going to walk through the transformer the way you would walk through a codebase you have just inherited: not by reading the paper's diagram top to bottom, but by following the data. A sequence of integers comes in. A sequence of probability distributions comes out. In between, the same block is applied over and over, and that block is made of only four kinds of operation: a lookup, a weighted average, a two-layer perceptron, and a normalization. Everything else in the field, from FlashAttention to mixture-of-experts to the KV cache, is an engineering decision about how to make one of those four cheaper. Chapter 8 will redo this walk counting FLOPs and bytes; here we count parameters and build intuition.

One housekeeping note. This chapter starts from token IDs. Chapter 2 explained how text becomes tokens; if you skipped it, Figure 3.1 is the one-paragraph version you need, and nothing later depends on more than that.

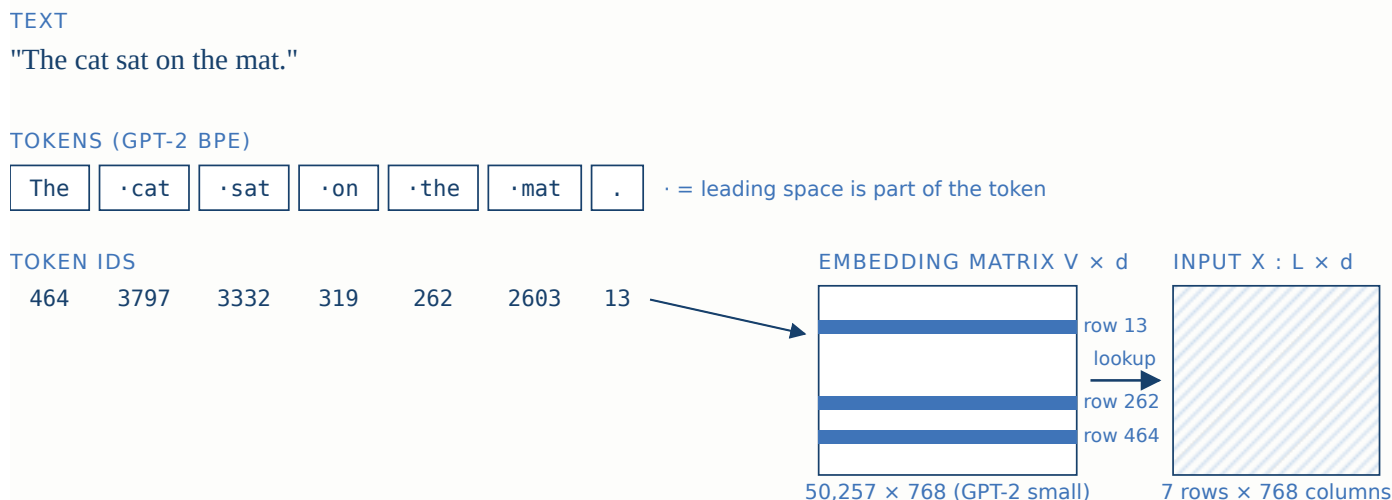


Figure 3.1 — From text to the transformer's input. The sentence is split into seven tokens by a BPE tokenizer (GPT-2's, here); the leading space belongs to the token, which is why `· cat` and `cat` are different vocabulary entries. Each token becomes an integer, and each integer selects one row of the embedding matrix. The transformer never sees text: it sees the 7×768 matrix on the right. Run `tiktoken.get_encoding("gpt2").encode("The cat sat on the mat.")` to reproduce the ids.

Hold on to three facts from that figure. First, the model's input is a matrix with one row per token and d columns, where d is the hidden size (768 for GPT-2 small, 4,096 for a 7B model). Second, the embedding matrix is just a lookup table with one row per vocabulary entry; it is a parameter matrix like any other, and with a 32,000-word vocabulary and $d = 4096$ it alone holds 131 million numbers. Third, and most important for this chapter: from here on, the row count L (sequence length) and the column count d never change. Every block reads an $L \times d$ matrix and writes an $L \times d$ matrix. That constraint is the single most useful thing to know about the architecture, and it is where we start.

3.1 The residual stream as a bus

If you have designed a hardware system or a message bus, you already have the right mental model for a transformer. There is a shared channel, one d -dimensional vector per token position, and a sequence of components attached to it. Each component reads the current state of the channel, computes something, and *adds* its result back. It does not replace the channel's contents. It contributes to them.

The original paper drew this as a stack of boxes with little "Add & Norm" labels, which makes the addition look like a detail.¹ It is not a detail; it is the architecture. Written as an equation, one transformer layer ℓ is nothing more than

$$x_{\ell+1} = x_{\ell} + \text{Attention}(x_{\ell}) + \text{MLP} \left(x_{\ell} + \text{Attention}(x_{\ell}) \right),$$

and the whole model is that line applied N times. Researchers at Anthropic gave the shared channel a name that has stuck, the *residual stream*, and the framing that goes with it: attention heads and MLPs are independent components that read from the stream and write to it, and the stream itself is the model's working memory for each token.²

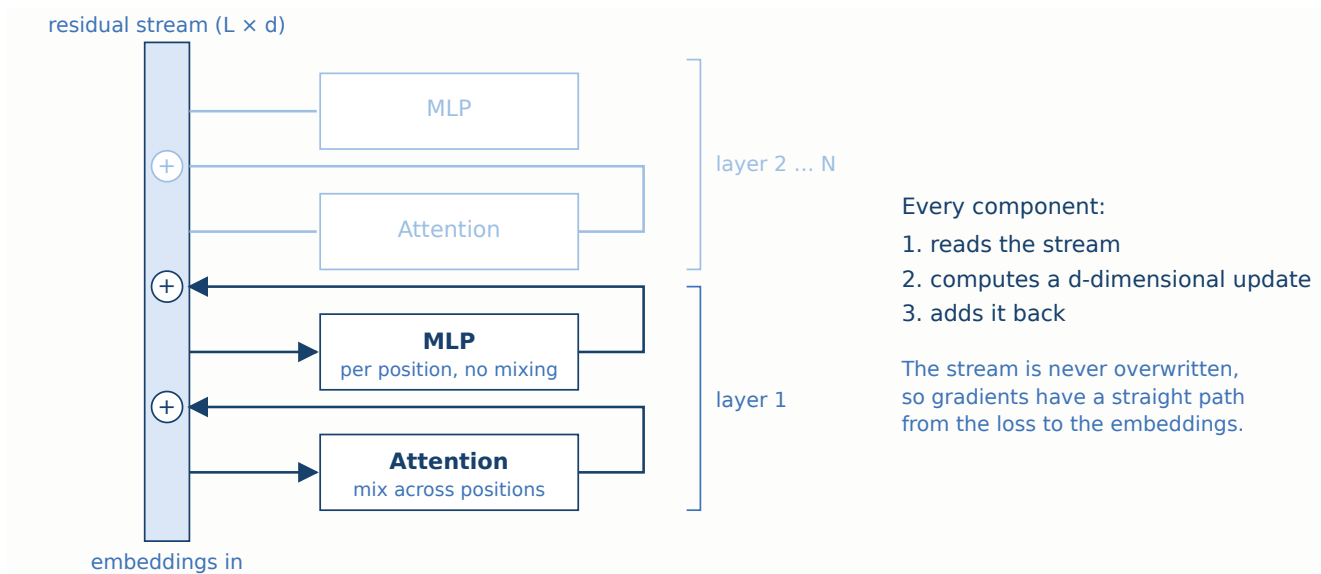


Figure 3.2 — The residual stream as a bus. Data flows bottom to top. Each attention or MLP component reads the whole stream, computes an update of the same shape, and adds it. Attention is the only component that moves information *between* positions; the MLP works on each position independently. Normalization layers are omitted here and added in Section 3.4.

Why does this matter for efficiency, which is what this book is about? Three reasons, and you will meet each of them again. The first is optimization: because the stream is added to rather than transformed, the gradient of the loss with respect to the embeddings has a path that passes through nothing but additions. A 32-layer network without residual connections is very hard to train; with them, it is routine. This is the same insight that made deep convolutional networks trainable in 2015, imported wholesale.³ The second is shape discipline: because every component consumes and produces $L \times d$, you can stack as many as you like, remove some, split them across GPUs at the boundaries, or recompute them during the backward pass, and nothing else has to change. Chapter 14 exploits that for pipeline parallelism and Chapter 15 for activation checkpointing. The third is that the hidden size d is the number every cost scales with. Attention weights are $d \times d$. MLP weights are $d \times 4d$ or thereabouts. Activations you must keep for the backward pass are $L \times d$ per layer. When someone tells you a model is "wider," they mean d went up, and almost every cost went up quadratically.

So a transformer is a bus with two kinds of component attached. Let us look at each.

3.2 Attention as a lookup over the sequence

Start with the problem attention solves. In the sentence "The cat sat on the mat," the representation of *sat* is, at the input, just the embedding of the token *sat*: the same vector it would have in "The dog sat," "We sat," or "sat" alone. To predict what comes next, the model needs the vector at that position to know *who* sat. Somehow, information from the position of *cat* has to flow into the position of *sat*. The MLP cannot do that; it looks at one position at a time. Attention is the component whose whole job is to move information between positions, and the way it does it is a soft version of something every engineer has written: a dictionary lookup.

In a dictionary you have keys, you have values, and you look things up with a query: the query is compared against each key, the matching key is found, and its value is returned. Attention does exactly this, with two changes. The

comparison is a dot product rather than an equality test, so it returns a score for *every* key rather than a yes/no. And rather than returning one value, it returns a weighted average of all the values, with weights given by the scores after a softmax. If one key matches strongly, the result is almost that key's value; if several match a bit, the result is a blend.

Concretely, take the input matrix X of shape $L \times d$ and multiply it by three learned matrices to get queries, keys, and values:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

each of shape $L \times d_k$. Every position now has a query ("what am I looking for?"), a key ("what do I offer?"), and a value ("what do I hand over if selected?"). The output is

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

Read it from the inside out. $QK^\top$ is an $L \times L$ matrix of scores: row i holds the dot product of position i 's query with every position's key. The softmax turns each row into weights that sum to one. Multiplying by V then takes, for each position, the weighted average of all value vectors. The result is again $L \times d_k$: one output vector per position, each a blend of the values of the positions it chose to attend to.

The $\sqrt{d_k}$ is easy to skip over and expensive to forget. Dot products of random vectors grow with their dimension: with $d_k = 128$, the scores start out with a standard deviation around 11, and a softmax over numbers that far apart is a hard argmax, with vanishing gradients everywhere except the winner. Dividing by $\sqrt{d_k}$ keeps the scores near unit variance so the softmax starts out soft and can learn.¹ Every from-scratch attention implementation that "trains but never learns anything" that we have debugged had dropped this one constant.

A worked example, with numbers

Let us do the lookup by hand for three tokens, *The*, *cat*, *sat*, with $d_k = 2$ so the numbers fit in a table. Suppose the projections have produced these queries and keys (identical here, to keep it simple) and these values:

$$Q = K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

The values are one-hot on purpose: that way the output for a position literally shows, as a vector, how much of each token's information it picked up. Table 3.1 follows the third token, *sat*, whose query is $(1, 1)$.

Table 3.1 — One attention lookup for the token *sat*, with $d_k = 2$. The query $(1, 1)$ matches its own key best, matches the other two equally, and the softmax turns those scores into a 25 / 25 / 50 blend of the three values.

KEY OF	SCORE $q \cdot k / \sqrt{2}$	SOFTMAX WEIGHT	VALUE	CONTRIBUTION
<i>The</i>	0.71	0.25	(1, 0, 0)	(0.25, 0, 0)
<i>cat</i>	0.71	0.25	(0, 1, 0)	(0, 0.25, 0)
		0.50	(0, 0, 1)	(0, 0, 0.50)

KEY OF	SCORE $q \cdot k / \sqrt{2}$	SOFTMAX WEIGHT	VALUE	CONTRIBUTION
<i>sat</i>	1.41	0.50	(0, 0, 1)	(0, 0, 0.50)
Output for <i>sat</i>				(0.25, 0.25, 0.50)

The output for *sat* is now a vector that is half its own information and a quarter each of *The* and *cat*. Information has moved across positions, which is what we wanted. In a trained model, of course, W_Q and W_K have learned to make the query of a verb match the key of its subject, so the weight on *cat* would be much higher than on *The*. That is what "learning attention patterns" means: learning the two projection matrices so that the right keys match the right queries.

The causal mask

There is one thing wrong with the example for a language model. When computing the output for *cat* (the second token), the lookup as written would consult the key of *sat*, which comes later. At training time the whole sentence is available, so nothing stops this; but at generation time, when the model is producing *sat*, it has not been written yet. If the model learned to peek ahead during training, it would be useless at inference. The fix is to add $-\infty$ to every score where the key position is after the query position before the softmax; $e^{-\infty} = 0$, so those weights vanish. The score matrix becomes lower-triangular. For our three tokens, the weights become

$$\begin{pmatrix} 1.00 & 0 & 0 \\ 0.33 & 0.67 & 0 \\ 0.25 & 0.25 & 0.50 \end{pmatrix},$$

and you can check that the first token attends only to itself, the second to itself and the first, and the third row is Table 3.1. This triangular structure is why the models in this book are called *causal* or *autoregressive*, and, as Chapter 16 will show, getting this mask right when several documents share a sequence is the most common bug in a training pipeline.

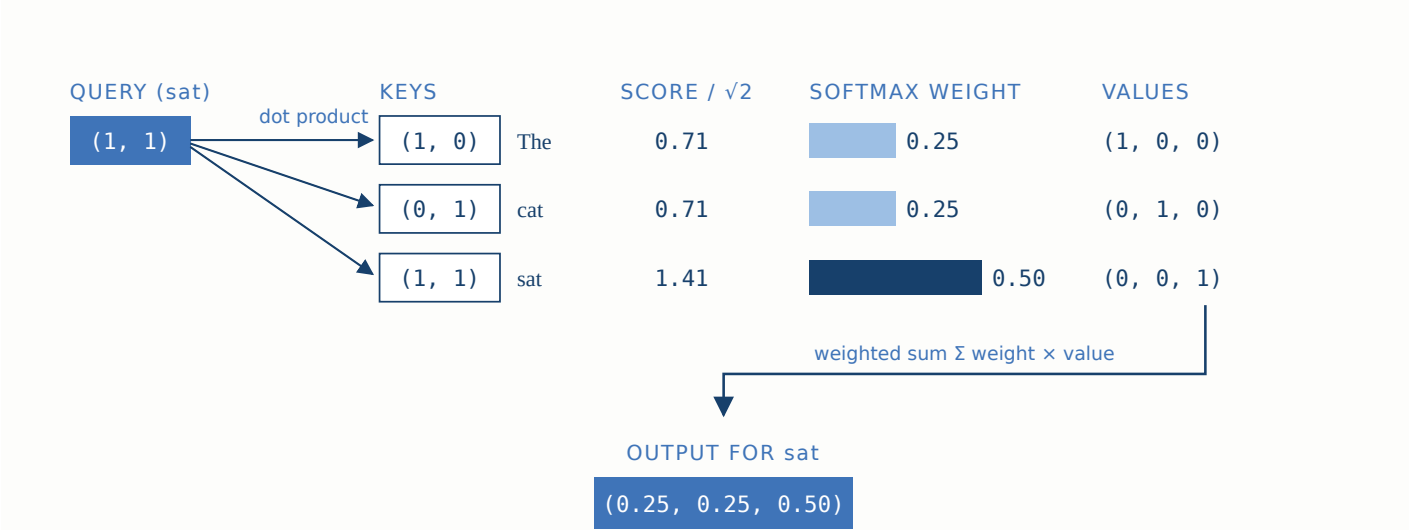


Figure 3.3 — Attention is a soft dictionary lookup. The query for *sat* is dotted with every key to produce scores; softmax turns the scores into weights that sum to one; the output is the weighted average of the values. Nothing is selected outright; everything is blended in proportion to how well it matched. The numbers are those of Table 3.1.

Many heads

One lookup per position can express one kind of relationship at a time: "find my subject," say. A sentence needs several at once: find the subject, find the previous occurrence of this word, find the closing bracket, find the noun this adjective modifies. The original paper's answer is to run several attention lookups in parallel, each with its own small W_Q, W_K, W_V , and concatenate the results.¹ These are the *heads*. With h heads, each head works in $d_k = d/h$ dimensions, so eight heads of 64 dimensions cost the same as one head of 512: the total projection is still $d \times d$ for each of Q, K, and V. After concatenation, one more $d \times d$ matrix, W_O , mixes the heads' outputs and writes the result back onto the residual stream. That gives the attention block exactly four square matrices, $4d^2$ parameters, a number we will use in Section 3.6.

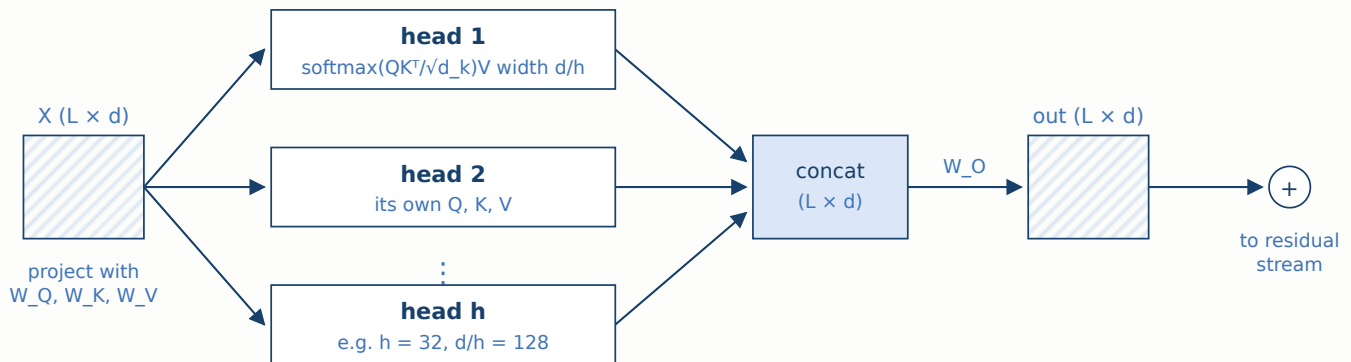


Figure 3.4 — Multi-head attention. The input is projected into h narrow heads, each running its own lookup in d/h dimensions. Their outputs are concatenated back to width d , mixed by W_O , and added to the stream. Total parameters: four $d \times d$ matrices, regardless of how many heads.

The whole thing in twenty lines

Here is causal multi-head self-attention written from scratch in PyTorch, with nothing hidden. Read it against the equation; every line corresponds to something above.

```
import torch, torch.nn as nn, torch.nn.functional as F

class CausalSelfAttention(nn.Module):
    def __init__(self, d, n_heads):
        super().__init__()
        self.h, self.dk = n_heads, d // n_heads
        self.qkv = nn.Linear(d, 3 * d, bias=False) # W_Q, W_K, W_V fused
        self.out = nn.Linear(d, d, bias=False)      # W_O

    def forward(self, x):                            # x: (B, L, d)
        B, L, d = x.shape
        q, k, v = self.qkv(x).split(d, dim=-1)
        # (B, L, d) -> (B, h, L, dk): one lookup per head
        q, k, v = (t.view(B, L, self.h, self.dk).transpose(1, 2) for t in (q, k,
v))
        scores = q @ k.transpose(-2, -1) / self.dk ** 0.5 # (B, h, L, L)
        mask = torch.triu(torch.ones(L, L, dtype=torch.bool), 1) # future
positions
```

```

scores = scores.masked_fill(mask, float("-inf"))
w = F.softmax(scores, dim=-1)
y = w @ v                                     # (B, h, L, dk)
y = y.transpose(1, 2).reshape(B, L, d)        # concat heads
return self.out(y)

```

Two things to notice about cost, which Chapter 13 will spend a whole chapter on. The `scores` tensor is $B \times h \times L \times L$: at $L = 8192$ and 32 heads, that is two billion numbers per sequence, and this naive implementation materializes all of them. FlashAttention exists to avoid ever writing that tensor to memory. And the matrix multiplications that produce Q, K, V and the output are linear in L , while the score computation is quadratic; at short contexts the projections dominate, at long contexts the L^2 term does. Where the crossover happens is the question that decides whether "long context" is cheap or ruinous for your model.

3.3 The MLP: where the knowledge lives

After attention has moved information between positions, the second component processes each position on its own. It is the plainest neural network there is, a two-layer perceptron applied identically to every row of the stream:

$$\text{MLP}(x) = W_{\text{down}} \sigma(W_{\text{up}} x),$$

where W_{up} expands from d to an intermediate size d_{ff} , σ is an elementwise nonlinearity, and W_{down} projects back to d . The original paper used $d_{\text{ff}} = 4d$ and a ReLU;¹ that 4× ratio survived, almost unquestioned, for years.

The MLP is easy to dismiss as the boring block, and it is where most of the parameters and most of the knowledge are. Count: two matrices of $d \times 4d$ make $8d^2$ parameters, against attention's $4d^2$. Two-thirds of every layer is MLP. And there is a useful way to see what those parameters are doing, due to Geva and colleagues: the MLP is itself a key-value memory.⁴ Each row of W_{up} is a key: the dot product with x measures how well the current token's state matches a pattern, and the nonlinearity zeroes out weak matches. Each column of W_{down} is the value written to the stream when its key fires. A 7B model has 32 layers $\times$ 11,008 such key-value pairs, and when people say a model "knows" that Lima is the capital of Peru, this is the kind of place that fact is stored: as a key that fires on the pattern "capital of Peru" and a value that pushes the stream toward "Lima."

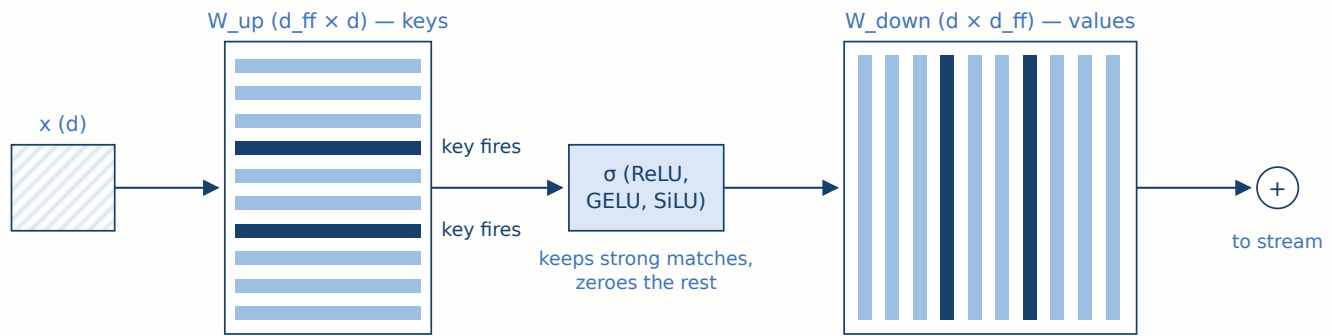


Figure 3.5 — The MLP as a key-value memory. Each row of W_{up} is a key matched against the token's current state; the nonlinearity keeps only the strong matches (dark rows); the corresponding columns of W_{down} (dark columns) are the values added to the residual stream. Two keys fired here, so two values were written.

Modern models have made one change here that you will see in every config: the *gated* MLP. Shazeer showed in 2020 that replacing $\sigma(W_{\text{up}}x)$ with $\sigma(W_{\text{gate}}x) \odot (W_{\text{up}}x)$, an elementwise product of a gate and an ungated projection, trains noticeably better for the same compute; with SiLU as the nonlinearity this is called SwiGLU.⁵ The cost is a third matrix. To keep the parameter count roughly constant, implementations shrink the intermediate size to about $\frac{8}{3}d$ and then round to a hardware-friendly multiple, which is exactly why Llama 2 7B has the odd-looking `intermediate_size: 11008` for $d = 4096$: $\frac{8}{3} \times 4096 = 10923$, rounded up to a multiple of 256.⁶ The gated MLP therefore has $3d d_{\text{ff}}$ parameters rather than $2d d_{\text{ff}}$, and this difference matters when you count.

```
class SwiGLU(nn.Module):
    def __init__(self, d, d_ff):
        super().__init__()
        self.gate = nn.Linear(d, d_ff, bias=False)  # W_gate
        self.up = nn.Linear(d, d_ff, bias=False)    # W_up
        self.down = nn.Linear(d_ff, d, bias=False)  # W_down

    def forward(self, x):
        # x: (B, L, d)
        return self.down(F.silu(self.gate(x)) * self.up(x))
```

Because the MLP treats every position independently, it is embarrassingly parallel across the sequence: a batch of B sequences of length L is just BL independent vectors going through the same two matrices. That is why the MLP is where GPUs are happiest (big, dense, regular matrix multiplications) and why, in Chapter 8, it will turn out to dominate the FLOP count at moderate sequence lengths even though attention gets all the attention.

3.4 Normalization and positional encodings

Two more pieces and the block is complete. Neither is glamorous, and each has caused more failed training runs than attention ever did.

Normalization: keeping the stream at a sane scale

Recall that every component adds to the residual stream and nothing ever subtracts. After thirty-two layers of additions, the vectors on the stream can be much larger than they were at the input, and their scale differs from layer to layer and from run to run. Matrix multiplications do not care about scale, but the softmax in attention and the nonlinearity in the MLP do: feed a softmax inputs ten times larger and it becomes an argmax; feed a ReLU inputs that have drifted negative and it outputs zeros forever. Normalization is the fix. Before a component reads the stream, you rescale each token's vector to a standard size, and the component sees well-behaved inputs no matter what has accumulated on the stream.

The original recipe was LayerNorm: subtract the mean of the vector, divide by its standard deviation, then multiply by a learned per-dimension gain and add a learned bias.⁷ Almost every model trained since 2022 uses RMSNorm instead, which drops the mean subtraction and the bias and just divides by the root-mean-square of the vector:⁸

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \odot g.$$

It is cheaper (one reduction over the vector instead of two, no bias), it trains as well or better, and it has d parameters per instance, which is small enough that we will ignore it when counting. The ϵ is there to avoid dividing by zero, and its value (10^{-5} or 10^{-6} , in the config as `rms_norm_eps`) actually matters in low precision; we will come back to that in Chapter 17.

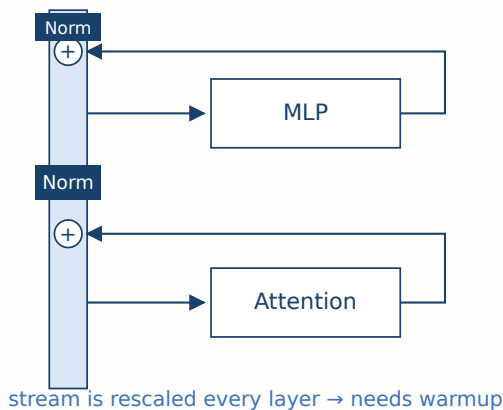
```
class RMSNorm(nn.Module):
    def __init__(self, d, eps=1e-6):
        super().__init__()
        self.g, self.eps = nn.Parameter(torch.ones(d)), eps

    def forward(self, x):
        # x: (... , d)
        rms = x.pow(2).mean(-1, keepdim=True).add(self.eps).rsqrt()
        return x * rms * self.g
```

Where you put the normalization turns out to matter more than which one you use. The 2017 paper placed it *after* the residual addition (post-norm): compute the component, add to the stream, normalize the stream. That means the stream itself is renormalized at every layer, which sounds tidy and in practice makes deep transformers fragile: the gradient no longer has a clean path through the additions, and training only works with a carefully tuned learning-rate warmup. Xiong and colleagues showed in 2020 that moving the normalization to the input of each component (pre-norm), so the stream itself is never rescaled, gives well-behaved gradients at initialization and removes the need for warmup entirely.⁹ GPT-2 had already made this move in 2019,¹⁰ and every model in Table 3.2 after the original uses pre-norm. Figure 3.6 shows the difference; the only thing that moves is the small norm box, and it is the difference between a model that trains and one that diverges at step 2,000.

Post-norm (2017)

$$x \leftarrow \text{Norm}(x + f(x))$$



Pre-norm (GPT-2 onward)

$$x \leftarrow x + f(\text{Norm}(x))$$

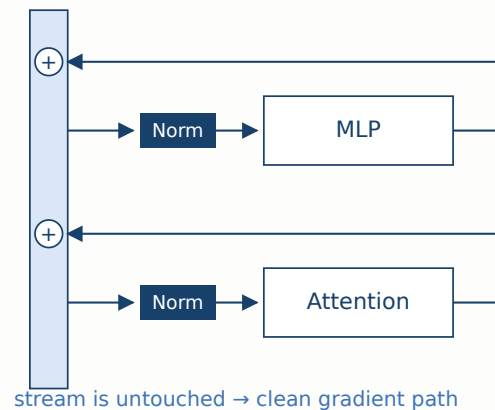


Figure 3.6 — Post-norm versus pre-norm. Left: the original design normalizes the residual stream itself after each addition. Right: modern models normalize only the copy that feeds each component, leaving the stream as a pure sum. Same parameters, same FLOPs, very different training stability. A final norm is applied to the stream before the output layer in both designs.

Positions: telling the model where each token is

Here is a fact about attention that surprises people the first time: it does not know the order of the tokens. Look at the equation again. Every query is compared with every key by a dot product, and the values are summed with the resulting weights. Nothing in that computation depends on which row came first. If you shuffle the input rows, the output rows shuffle identically. The MLP is per-position, so it does not know either. A transformer with no position information would treat "the cat sat on the mat" and "the mat sat on the cat" as the same bag of tokens.

So position has to be injected. The 2017 solution was to add to each token's embedding a fixed vector that encodes its position, built from sines and cosines at geometrically spaced frequencies:¹

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right).$$

The first pair of dimensions cycles every few positions; the last pair barely changes across thousands of positions, so together they form something like a binary clock: fast digits and slow digits, with a unique reading at every position and the property that the offset between two positions can be read off as a rotation. GPT-2 replaced this with a learned table of 1,024 position vectors, one per allowed position, which works just as well and cannot represent position 1,025 at all.¹⁰

Both approaches add position to the token embedding once, at the bottom of the stack, and hope it survives thirty-two layers of additions. Rotary position embeddings (RoPE), introduced by Su and colleagues in 2021 and used by every model in Table 3.2 from Llama onward, take a different route: instead of adding a position vector to the stream, they rotate the query and key vectors inside each attention head by an angle proportional to the position, using the same geometric ladder of frequencies as the sinusoidal scheme.¹¹ The elegant consequence is that the dot product between a query at position m and a key at position n then depends only on the difference $m - n$, so attention sees *relative* position directly, in every layer, without anything having to be stored on the stream. RoPE adds no parameters. It does add a config field, `rope_theta`, which is the 10,000 in the equation above; raising it (Llama 3 uses 500,000)

stretches the slow frequencies so the model can tell positions apart over longer contexts, and that knob is the starting point of every context-extension recipe in Chapter 21.

$\sin(\text{pos} / 10000^{(2i/d)})$ for three dimension pairs i , positions 0 ... 60

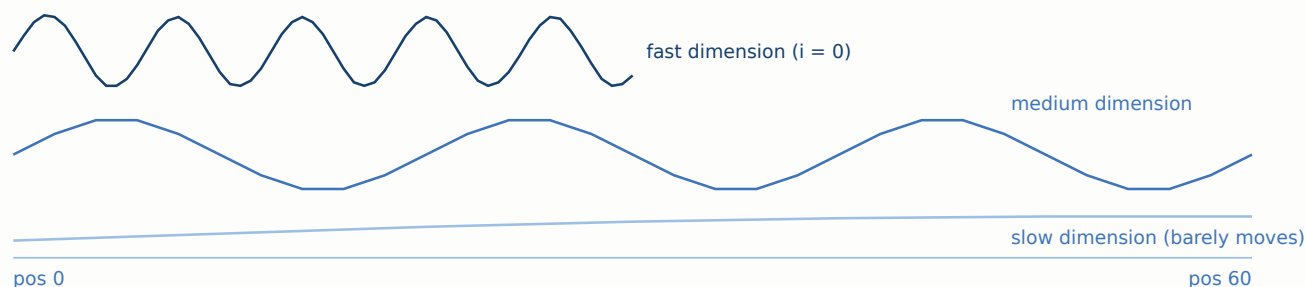


Figure 3.7 — Sinusoidal position signals. Low-index dimensions cycle every few positions and distinguish neighbors; high-index dimensions change slowly and distinguish distant regions. Together they give every position a unique code in which relative offsets appear as rotations. RoPE uses the same ladder of frequencies but applies it as a rotation of Q and K rather than as an addition to the embedding.

3.5 Why decoder-only won

We have been describing a single stack of blocks. The 2017 transformer was two of them, and the story of how it became one is worth ten minutes because it explains why the models you will train look the way they do.

The original paper was about machine translation, and translation has an input sentence and an output sentence. So the architecture had an *encoder*, a stack of six blocks that read the source sentence with full bidirectional attention (every token sees every other, no causal mask, since the whole source is known), and a *decoder*, a stack of six blocks that generates the target one token at a time. Each decoder block has three components rather than two: causal self-attention over the target so far, then *cross-attention* whose queries come from the decoder but whose keys and values come from the encoder's output, and then the MLP. Cross-attention is the bridge: it is how the token being generated in German looks up the relevant words in the English source. Figure 3.8 is the whole thing; you will recognize every box from the previous sections.

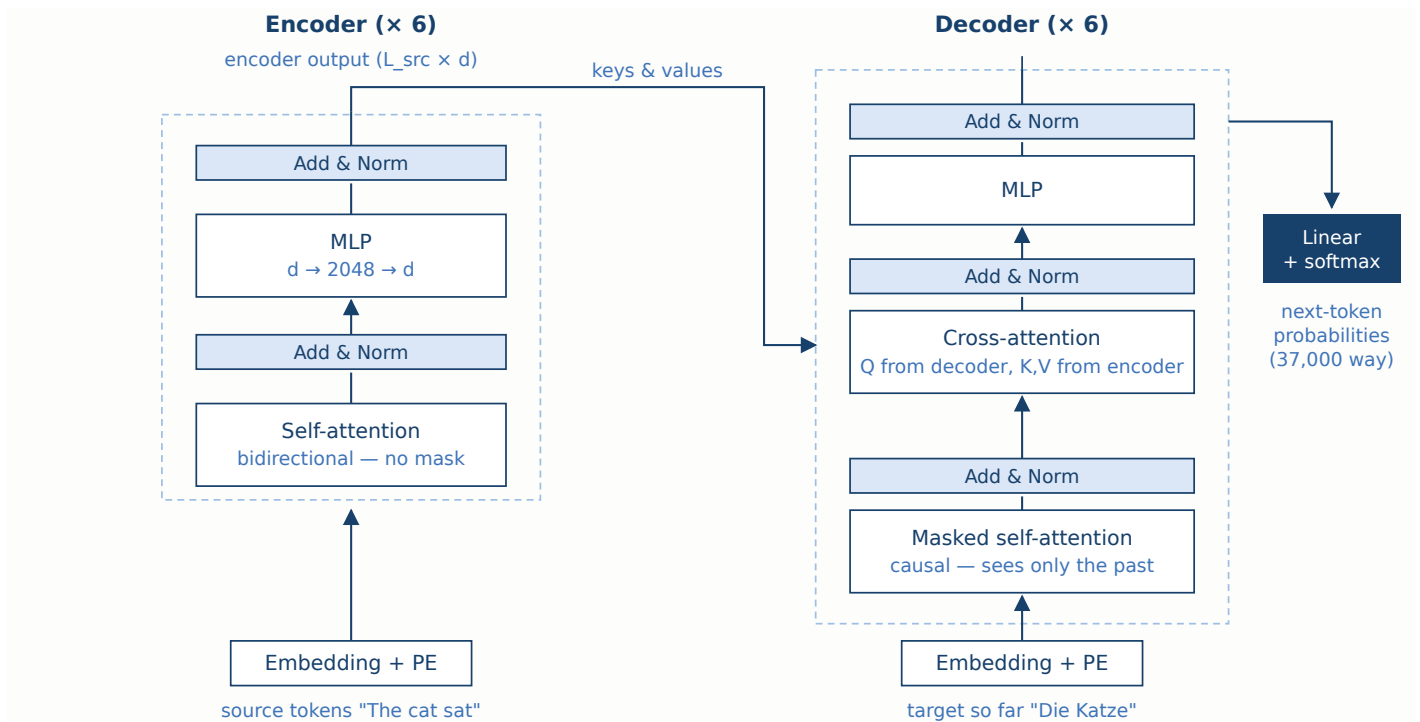


Figure 3.8 — The original transformer (Vaswani et al., 2017). The encoder reads the whole source with bidirectional attention. The decoder generates the target token by token with causal self-attention, consults the encoder output through cross-attention, and ends in a softmax over the vocabulary. Six blocks each, $d = 512$, 8 heads, $d_{ff} = 2048$, 65 million parameters in the base configuration.

Three descendants took three different pieces of this design. BERT kept only the encoder and trained it to fill in masked words, which produced superb representations for classification but a model that cannot generate text.¹² T5 kept the whole encoder-decoder and cast every task as text-to-text.¹³ GPT kept only the decoder, deleted the cross-attention it no longer had anything to attend to, and trained it on the simplest objective there is: predict the next token.¹⁴ For a few years all three coexisted. Today, every model in this book, and essentially every model you would consider training from scratch, is GPT's shape. The reasons are worth understanding because they are efficiency reasons.

The first is that a decoder-only model gets a training signal from every token. In one forward pass over a 4,096-token sequence, it makes 4,096 predictions and receives 4,096 gradients. An encoder-decoder trained on translation gets signal only from the target side; a masked encoder like BERT gets signal only from the 15% of tokens it masked. Per FLOP spent, next-token prediction extracts several times more learning from the same data, and when data and compute are the budget, that decides it.

The second is that generation is native. A decoder trained to predict the next token is already a text generator; there is no gap between the pretraining objective and the way the model is used. This is also why the KV cache of Section 3.7 is the central inference structure: generation is the workload, and the architecture was trained for it.

The third is uniformity, which is the one an engineer should care about most. A decoder-only model is one block repeated N times, with identical shapes throughout. That makes it trivial to scale in depth, easy to split across GPUs at any layer boundary, and simple to reason about: everything in Chapters 13 through 17 assumes a stack of identical blocks. An encoder-decoder has two stacks with different attention patterns, different sequence lengths, and cross-attention wiring between them, and every one of those systems techniques becomes a special case. A careful

comparison in 2022 found that, given the choice of architecture and objective at equal compute, causal decoder-only models trained on plain next-token prediction generalized best to unseen tasks,¹⁵ and the field has not looked back.

So the block you will actually train is the 2017 decoder block with the cross-attention removed and four substitutions: pre-norm instead of post-norm, RMSNorm instead of LayerNorm, RoPE instead of sinusoidal or learned positions, and a gated SwiGLU MLP instead of ReLU. Table 3.2 tracks these substitutions across the models that made them. Every row after the first is the same diagram as Figure 3.9 with different constants.

Table 3.2 — How the block changed from 2017 to today. Every model below the first line is decoder-only. The four substitutions (norm placement, norm type, positions, activation) had all happened by 2023; since then the changes have been in attention (GQA, MLA) and in the MLP (mixture-of-experts), both aimed at cost rather than quality.

MODEL	STACK	NORM	POSITIONS	MLP	ATTENTION	SIZE
Transformer (2017) ¹	Enc + dec	Post-LN	Sinusoidal	ReLU, 4d	MHA, 8 heads	65M / 213M
GPT-2 (2019) ¹⁰	Decoder	Pre-LN	Learned (1,024)	GELU, 4d	MHA	117M – 1.5B
Llama 2 (2023) ⁶	Decoder	Pre-RMSNorm	RoPE, $\theta = 10^4$	SwiGLU, $\sim 8/3$ d	MHA (7B, 13B), GQA (70B)	7B – 70B
Llama 3 (2024) ¹⁶	Decoder	Pre-RMSNorm	RoPE, $\theta = 5 \cdot 10^5$	SwiGLU	GQA, 8 KV heads	8B – 405B
DeepSeek-V3 (2024) ¹⁷	Decoder	Pre-RMSNorm	RoPE (decoupled)	SwiGLU, MoE (256 experts)	MLA (latent KV)	671B total, 37B active

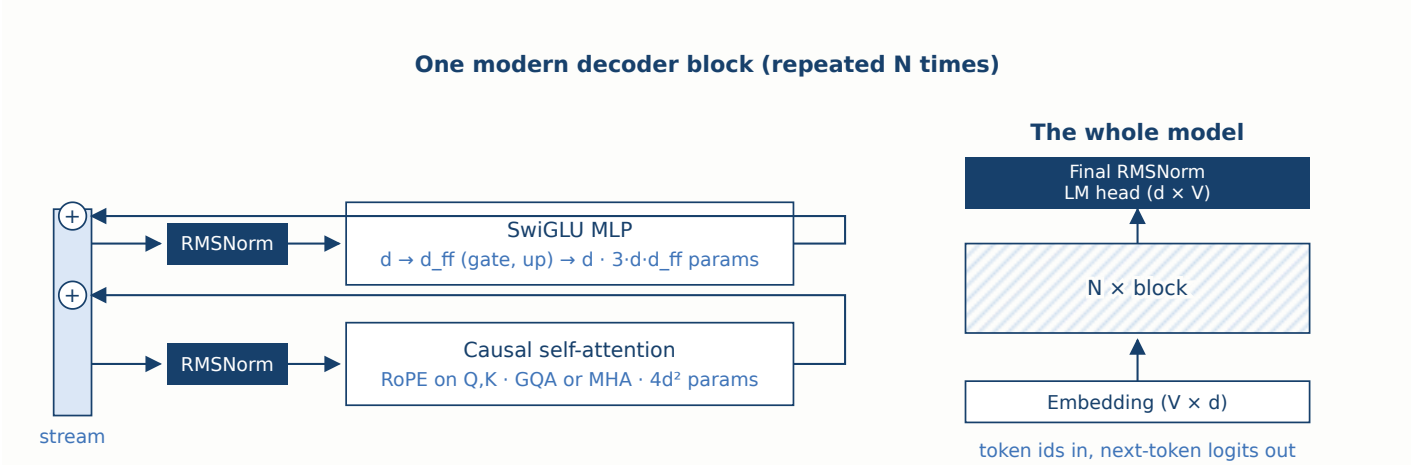


Figure 3.9 — What you will actually train. Left: one decoder block as used by Llama-family models. Right: the model is an embedding table, N copies of the block, a final norm, and an output projection back to the vocabulary. Compare with Figure 3.8: the cross-attention and the second stack are gone, and four components have been swapped.

3.6 Counting the parameters by hand

You now know every matrix in the model, so you can count them, and it is worth doing once by hand because the count is what memory, cost, and scaling laws are all computed from. Let us do it for the 2017 base model, where we can check our answer against the paper, and then for a 7B decoder, where we can check against the file size on disk.

The original Transformer base

The base configuration has $d = 512$, $d_{\text{ff}} = 2048$, six encoder and six decoder layers, and a shared vocabulary of about 37,000 byte-pair tokens.¹ An encoder layer has one attention block (four $d \times d$ matrices) and one ReLU MLP (two $d \times d_{\text{ff}}$ matrices):

$$P_{\text{enc}} = 4d^2 + 2d d_{\text{ff}} = 4(512)^2 + 2(512)(2048) = 1,048,576 + 2,097,152 \approx 3.15\text{M}.$$

A decoder layer has two attention blocks (self and cross) and the same MLP, so $P_{\text{dec}} = 8d^2 + 2d d_{\text{ff}} \approx 4.19\text{M}$. The embedding table is $37,000 \times 512 \approx 18.9\text{M}$, and the paper shares it between the encoder input, the decoder input, and the output softmax, so it is counted once. Putting it together:

$$P \approx 6 \times 3.15\text{M} + 6 \times 4.19\text{M} + 18.9\text{M} \approx 63\text{M},$$

and the paper reports 65M. The 2M gap is the biases, the LayerNorm gains and biases, and rounding of the vocabulary size: exactly the small terms we chose to ignore. That the count lands within 3% from five integers is the point. Notice also where the parameters are: 30% in the embedding table for this small model. That fraction shrinks fast as d and N grow, which is why scaling-law papers usually count "non-embedding" parameters and why, for a 7B model, the embedding is a rounding error.

A 7B decoder

Now Llama 2 7B, whose config is the one this chapter opened with: $d = 4096$, $N = 32$, $d_{\text{ff}} = 11008$, $V = 32000$, gated MLP, no biases, and separate (untied) input embedding and output head.⁶ Per layer:

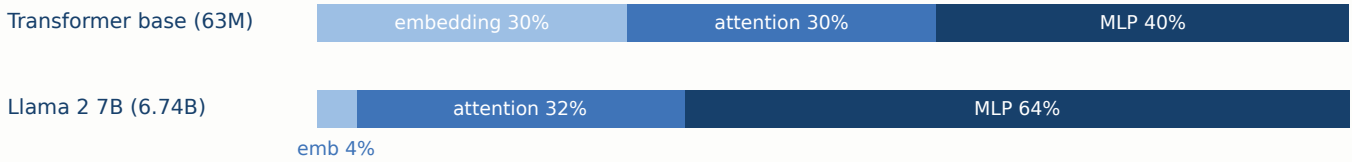
$$P_{\text{layer}} = \underbrace{4d^2}_{\text{attention}} + \underbrace{3d d_{\text{ff}}}_{\text{SwiGLU}} = 67.1\text{M} + 135.3\text{M} = 202.4\text{M},$$

and for the model:

$$P = 32 \times 202.4\text{M} + \underbrace{2 \times 32000 \times 4096}_{\text{embedding} + \text{LM head}} = 6.476\text{B} + 0.262\text{B} = 6.74\text{B}.$$

The released checkpoint has 6,738,415,616 parameters. Our count, ignoring the 65 RMSNorm vectors, is 6,738,149,376; the difference is 266,240, which is $65 \times 4,096$. Every number is accounted for.

Where the parameters are



Rule of thumb for a dense decoder with SwiGLU: $P \approx N \cdot (4d^2 + 3d \cdot d_{\text{ff}}) + 2Vd$. With $d_{\text{ff}} \approx 8/3 d$ this is $\approx 12 N d^2 + 2Vd$.

Figure 3.10 — Parameter breakdown. In the small 2017 model the vocabulary table is a third of the parameters; in a 7B decoder it is 4%, and the MLP holds nearly two-thirds. The $12Nd^2$ approximation for non-embedding parameters is the one used by Kaplan and colleagues in the first scaling-law paper,¹⁸ and it is accurate to a few percent for any standard dense decoder.

Here is the count as a function you can run on any config. It reads the five fields that matter and handles the two variants you will meet, grouped-query attention (fewer key and value heads than query heads, which shrinks W_K and W_V) and tied embeddings (one table shared between input and output):

```
def count_params(cfg):
    d, N, V = cfg["hidden_size"], cfg["num_hidden_layers"], cfg["vocab_size"]
    d_ff = cfg["intermediate_size"]
    h = cfg["num_attention_heads"]
    h_kv = cfg.get("num_key_value_heads", h) # GQA: fewer K,V heads
    d_head = d // h
    attn = d * (h * d_head) * 2 + d * (h_kv * d_head) * 2 # W_Q, W_O, W_K, W_V
    mlp = 3 * d * d_ff # gate, up, down
    emb = V * d * (1 if cfg.get("tie_word_embeddings") else 2)
    return N * (attn + mlp) + emb

llama2_7b = dict(hidden_size=4096, num_hidden_layers=32, vocab_size=32000,
                 intermediate_size=11008, num_attention_heads=32)
print(f"{count_params(llama2_7b)/1e9:.2f} B") # 6.74 B
```

Two immediate uses. Multiply by two bytes and you have the checkpoint size in BF16: 13.5 GB for this model, which is what you will see on disk. Multiply by six and you have the FLOPs per training token, $6P \approx 40$ GFLOP here, which Chapter 6 will use to price a run and Chapter 8 will derive properly. Everything downstream starts from this count.

3.7 The KV cache, in one page

This chapter is about training, but one inference structure is so tied to the architecture, and so decisive for the design choices in Table 3.2, that it belongs here.

When the model generates text, it produces one token, appends it to the sequence, and runs again to produce the next. Look at what attention needs at each step. To compute the output for the newest position, it needs that position's query, and the keys and values of *every* previous position. Those previous keys and values were already computed on the

previous step, and, because each token's K and V depend only on that token's own stream state, they have not changed. Recomputing them every step would make generating a sequence of length L cost $O(L^2)$ forward passes' worth of work. So you keep them. The *KV cache* is exactly that: for every layer and every head, the key and value vectors of every token seen so far, stored so that each new token costs one query against a growing table rather than a recomputation of the whole prefix.¹⁹

Generation with a KV cache (one layer, one head shown)

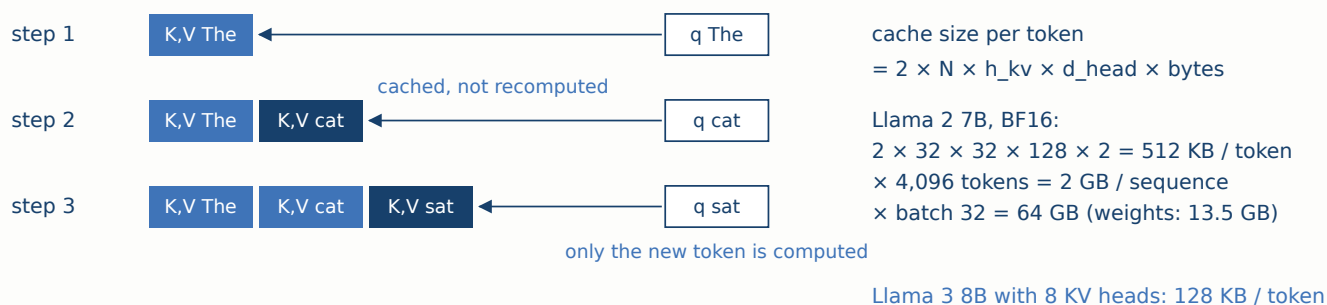


Figure 3.11 — The KV cache. Each generation step appends one key and one value per layer per head and runs one query against the whole table. The cache grows linearly with context and with batch size, and for a 7B model at 4,096 tokens it exceeds the weights once the batch is larger than about seven sequences. Grouped-query attention shrinks it by the ratio of query heads to KV heads.

The size of the cache follows directly from the shapes in Section 3.2. Per token, per layer, you store one key and one value vector for each KV head, each of dimension d_{head} :

$$\text{bytes per token} = 2 \times N \times h_{kv} \times d_{head} \times \text{bytes per element}.$$

For Llama 2 7B in BF16 that is $2 \times 32 \times 32 \times 128 \times 2 = 512 \text{ KB}$ per token, so 2 GB for one 4,096-token sequence and 64 GB for a batch of 32: five times the size of the weights. This is why serving is a memory problem, and it is the entire motivation for the attention variants in Table 3.2. Multi-query attention (one shared K and V head) and grouped-query attention (a few) exist to shrink h_{kv} .²⁰ Llama 3 8B keeps 32 query heads but only 8 KV heads, cutting the cache to 128 KB per token.¹⁶ DeepSeek's multi-head latent attention goes further and stores a compressed latent vector per token instead of the keys and values themselves.¹⁷ Chapter 13 covers all three. The point for now is that a decision about attention heads, made at architecture time and irreversible after training, sets the inference memory of the model for its whole life.

Training is different, and it is worth being explicit about how. During training the whole sequence is present at once, so attention for all positions is computed in one shot with the causal mask, and there is no cache: the keys and values are just intermediate activations. What training must store instead is every activation needed for the backward pass, at every layer, for every token in the batch, and that is a far larger quantity than the KV cache; Chapter 8 will show it is the dominant term in training memory, and Chapter 15 will show how to trade compute to shrink it. So the same architecture is memory-bound in two different ways depending on which direction the data flows: forward-only, the cache; forward-and-backward, the activations.

3.8 Practical considerations

Everything above is clean. Real configs and real implementations are not, and the gap between the two is where a surprising amount of engineering time goes. This section is the list we wish someone had handed us.

Reading a config without being misled

Start with the head geometry. `num_attention_heads` is the number of query heads; `num_key_value_heads`, if present and smaller, means grouped-query attention, and the parameter count and the KV cache both change (the function in Section 3.6 handles it). The head dimension is not usually stated: it is `hidden_size / num_attention_heads`, and it is 128 in almost every model since 2023, because that is the width at which attention kernels are fastest on current GPUs. If you are designing a model and pick a head dimension of 96 or 160, expect to lose throughput for no gain in quality.

Then the MLP. `intermediate_size` is d_{ff} , and whether it means two matrices or three depends on `hidden_act: silu` almost always means SwiGLU with three, `gelu` usually means the plain two-matrix MLP. Get this wrong and your parameter count is off by a third of the MLP, which is a fifth of the model.

The vocabulary is where the real surprises are. `vocab_size` is frequently padded to a multiple of 64, 128, or 256 for tensor-core efficiency and for even splitting under tensor parallelism, so it may not match the tokenizer's actual vocabulary; Llama 3's 128,256 is 128,000 real tokens plus 256 reserved. `tie_word_embeddings` tells you whether the input table and the output head are the same matrix; small models tie them to save parameters, large ones usually do not, and the difference is a full $V \times d$. And a large vocabulary shifts the balance of the model in a way that matters for multilingual work: with $V = 128,256$ and $d = 4096$, the embedding and head together are a billion parameters, an eighth of an 8B model, and the output softmax over 128K entries is a real fraction of the FLOPs per token. Chapter 11 will make the case that this is a good trade for Spanish and Portuguese, but it is a trade.

Finally, positions. `max_position_embeddings` is a hard limit for models with learned positions and a soft one for RoPE models, which will run at longer lengths but with degrading quality past what they were trained on. `rope_theta` is the base frequency; when you see it change between two versions of a model (10,000 to 500,000 between Llama 2 and Llama 3), the context length changed with it. Neither field adds parameters; both change behavior.

Common pitfalls

Counting the embedding twice, or the LM head not at all.

If your count is off by exactly $V \times d$, this is why. Check `tie_word_embeddings`, and remember that the final norm and head are outside the N repeated blocks.

Dropping the $1/\sqrt{d_k}$.

The model trains, the loss goes down slowly, and it never gets good. The softmax is saturated from step one and attention cannot learn patterns. This is the single most common bug in from-scratch implementations.

Masking with $-\infty$ in half precision.

`float("-inf")` in FP16 or BF16 can produce NaNs the moment a full row is masked (softmax of all $-\infty$ is $0/0$), which happens as soon as you pad. Use a large finite negative like -10^4 , or better, pass a boolean mask to a fused attention kernel and let it handle the arithmetic.

Off-by-one in the causal mask.

A position must be allowed to attend to itself. `torch.triu(ones, diagonal=1)` masks strictly future positions; `diagonal=0` masks the present as well and the first token attends to nothing, giving NaNs in the first row.

Assuming (L, B, d) when the tensor is (B, L, d) .

Older PyTorch code, and the original `nn.Transformer`, put the sequence dimension first. Everything in Hugging Face and almost everything written since 2021 puts batch first. Mixing them does not crash; it silently attends across the batch.

Post-norm without warmup.

If you implement the 2017 block faithfully and use a modern learning rate, it will diverge in the first few hundred steps. Use pre-norm. If you must use post-norm, warm the learning rate up over several thousand steps, as the original did.

Forgetting that RoPE is applied to Q and K only.

Not to V , and not to the residual stream. Applying it to V corrupts the values with position information they should not carry; the model still trains and is subtly worse, which is the worst kind of bug.

Treating heads as independent features.

After W_O , the heads are mixed; there is no "head 3's output" on the residual stream. Interpretability work that looks at individual heads reads them *before* the output projection, for exactly this reason.

Materializing the $L \times L$ score matrix at long context.

The twenty-line attention in Section 3.2 allocates $B \times h \times L \times L$ floats. At $L = 32,768$ with 32 heads and batch 1, that is 137 GB in FP32. Use `F.scaled_dot_product_attention` or FlashAttention for anything beyond a toy.

3.9 Where this shows up later

This chapter was a map, and almost every later chapter is a road on it. The parameter count of Section 3.6 becomes the FLOP count in Chapter 8, where the rule that a training step costs $6P$ FLOPs per token is derived from the same matrices we counted, and where the activations we said training must store are measured layer by layer. Chapter 6 uses the same count to put a dollar figure on a run before it starts.

The vocabulary table's share of the model, and the cost of the output softmax, are the subject of Chapter 11, which argues that for Latin American languages the right vocabulary is larger than the English-centric default and shows what

that does to the numbers in Figure 3.10. The architecture substitutions of Table 3.2 are each a decision in Chapter 12: RoPE's base frequency, mixture-of-experts as a way of growing the MLP's key-value memory without growing the compute per token, the depth-versus-width trade-off, and the muP parametrization that lets hyperparameters tuned on a small d transfer to a large one.

The quadratic term in attention, the score matrix we warned about materializing, and the KV cache of Section 3.7 are together the subject of Chapter 13, which covers FlashAttention (never write the score matrix), grouped-query and latent attention (shrink the cache), and the sliding-window and linear variants that change the cost curve outright. Chapter 14 splits the model across GPUs, and it will do so exactly at the seams this chapter exposed: heads across tensor-parallel ranks, layers across pipeline stages, sequence positions across context-parallel ranks. Chapter 16 comes back to the causal mask when several documents share one sequence, and Chapter 21 comes back to `rope_theta` when the context needs to grow after pretraining.

3.10 Summary

A transformer is a residual stream, one d -dimensional vector per token, with two kinds of component attached to it, repeated N times. Attention moves information between positions by a soft dictionary lookup: queries against keys, softmax, weighted sum of values, with a causal mask so no position sees its future and a $1/\sqrt{d_k}$ so the softmax stays soft. It costs $4d^2$ parameters per layer and a quadratic amount of compute in the sequence length. The MLP processes each position alone through an expand-and-contract pair of matrices, holds two-thirds of the parameters, and is where the model's knowledge lives as key-value pairs. Normalization keeps the stream at a usable scale and is placed before each component, not after; position is injected by rotating queries and keys rather than by adding to the stream. The original 2017 design had two stacks and cross-attention between them; the decoder-only stack won because it learns from every token, generates natively, and is a uniform block that scales and shards cleanly.

Five integers from a config give you the whole cost picture. Parameters are $N(4d^2 + 3 d d_{\text{ff}}) + 2Vd$: 6.74B for Llama 2 7B, exact to the RMSNorm vectors. Bytes on disk are two per parameter. FLOPs per training token are six per parameter. The KV cache is $2Nh_{\text{kv}}d_{\text{head}}$ elements per generated token, and it, not the weights, is what fills a serving GPU. If you can produce those four numbers from a config, you are ready for Part II, where we start asking how to make each of them smaller.

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *NeurIPS*. arXiv:1706.03762. Base model: $d_{\text{model}} = 512$, 6 + 6 layers, 8 heads, $d_{\text{ff}} = 2048$, 65M parameters, trained for 100K steps (about 12 hours) on 8 P100 GPUs; the big model has 213M parameters.
2. Elhage, N., Nanda, N., Olsson, C., et al. (2021). A Mathematical Framework for Transformer Circuits. *Transformer Circuits Thread*, Anthropic. Introduces the residual-stream view in which attention heads and MLPs read from and write to a shared communication channel.

3. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*. arXiv:1512.03385.
4. Geva, M., Schuster, R., Berant, J., & Levy, O. (2021). Transformer Feed-Forward Layers Are Key-Value Memories. *EMNLP*. arXiv:2012.14913.
5. Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202. Reports that SwiGLU and related gated units improve perplexity and downstream quality at matched parameters and compute, with the intermediate size reduced by a factor of 2/3.
6. Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288. The 7B configuration: 32 layers, $d = 4096$, 32 heads, $d_{\text{ff}} = 11008$, vocabulary 32,000, RMSNorm, SwiGLU, RoPE, context 4,096; GQA is used only in the 34B and 70B models.
7. Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer Normalization. arXiv:1607.06450.
8. Zhang, B., & Sennrich, R. (2019). Root Mean Square Layer Normalization. *NeurIPS*. arXiv:1910.07467.
9. Xiong, R., Yang, Y., He, D., et al. (2020). On Layer Normalization in the Transformer Architecture. *ICML*. arXiv:2002.04745. Shows that pre-LN transformers have well-behaved gradients at initialization and can be trained without the learning-rate warmup that post-LN requires.
10. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI. GPT-2 moves layer normalization to the input of each sub-block, uses learned positions with a 1,024-token context, and a 50,257-token byte-level BPE vocabulary; sizes from 117M to 1.5B parameters.
11. Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.
12. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL*. arXiv:1810.04805.
13. Raffel, C., Shazeer, N., Roberts, A., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 21(140). arXiv:1910.10683.
14. Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI.
15. Wang, T., Roberts, A., Hesslow, D., et al. (2022). What Language Model Architecture and Pretraining Objective Work Best for Zero-Shot Generalization? *ICML*. arXiv:2204.05832.
16. Meta AI (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Llama 3 8B: 32 layers, $d = 4096$, 32 query heads and 8 key-value heads, $d_{\text{ff}} = 14336$, vocabulary 128,256, RoPE base 500,000.
17. DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437. Multi-head latent attention compresses keys and values into a low-rank latent per token; the MoE MLP has 671B total and 37B activated parameters.
18. Kaplan, J., McCandlish, S., Henighan, T., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361. Uses $N \approx 12 n_{\text{layer}} d_{\text{model}}^2$ for non-embedding parameters.
19. Pope, R., Douglas, S., Chowdhery, A., et al. (2022). Efficiently Scaling Transformer Inference. arXiv:2211.05102. Analyzes KV-cache memory and bandwidth as the dominant cost of decoding at scale.
20. Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150 (multi-query attention); Ainslie, J., Lee-Thorp, J., de Jong, M., et al. (2023). GQA: Training Generalized Multi-Query

Transformer Models from Multi-Head Checkpoints. *EMNLP*. arXiv:2305.13245.